

Spécification de fonction

Pré-condition, post-condition, contrat de fonction et tests de propriété

Sommaire

PARTIE I · INTRODUCTION

01	Le problème posé	3
02	L'objectif et la démarche	4

PARTIE II · LA SPÉCIFICATION COMME CONTRAT

03	Pré-condition et post-condition	5
04	Le contrat de fonction	5
05	Spécification et implémentation	6
06	Spécifier la division euclidienne	6
07	Qui perd au contrat ?	8

PARTIE III · LES TESTS DE PROPRIÉTÉ

08	Principe et mise en place	9
09	Deux règles d'écriture	9
10	Précisions sur assert	10
11	Le format des examens CC3 et CC4	11

PARTIE IV · APPLICATION : MAXIMUM D'UNE LISTE

12	Maximum d'une liste	13
13	Pourquoi les deux parties sont nécessaires	14

PARTIE V · BILAN

14	Comparaison avec les autres approches	16
15	Un double objectif	17

POUR RÉVISER

·	Glossaire des définitions	18
·	Synthèse	18

01 Le problème posé

diapos 3–4

CADRE DU PROBLÈME

Le cadre d'étude proposé est celui des **fonctions**. Il est cependant **généralisable** à tout code présentant une ou plusieurs entrées, et une ou plusieurs sorties : programme principal, portion de code.

On exclut les mêmes situations qu'au CM 1 : les **interactions avec l'utilisateur** (les entrées sont données simultanément, les sorties recueillies simultanément) et les **effets de bord non testables** comme l'affichage.

L'exemple qui sert de fil rouge

FONCTION DONNÉE

```
def division_euclidienne(a,b):
    q,r = 0,a
    while r>b:
        q,r = q+1,r-b
    return q,r
```

Question 1. a) Déterminer les valeurs de retour de cette fonction dans les cas donnés. b) Cette fonction effectue-t-elle bien la division euclidienne de a par b ?

PROPOSITION DE CORRECTION · QUESTION 1A

A	B	RETOUR (q, r)	VÉRIFICATION
10	3	(3, 1)	$10 = 3 \times 3 + 1$, et $0 \leq 1 < 3$ ✓
17	5	(3, 2)	$17 = 5 \times 3 + 2$, et $0 \leq 2 < 5$ ✓
3	10	(0, 3)	$3 = 10 \times 0 + 3$, et $0 \leq 3 < 10$ ✓
14	7	(1, 7)	$14 = 7 \times 1 + 7$ ✓ mais $7 < 7$ est faux ✗
6	0	-	boucle infinie : r ne diminue jamais ✗

PROPOSITION DE CORRECTION · QUESTION 1B

Non, et deux défauts distincts apparaissent :

- la condition de boucle est $r > b$ au lieu de $r \geq b$: quand b divise a , la boucle s'arrête un tour trop tôt et renvoie un reste égal à b — le cas (14, 7) ;
- rien n'interdit $b = 0$, pour lequel la fonction **ne termine pas** — le cas (6, 0).

Le premier est un défaut d'**implémentation**, le second un défaut de **spécification** : personne n'a jamais écrit que b devait être non nul. C'est exactement la distinction que le chapitre va installer.

Notez la nature du problème : la fonction n'a pas « planté ». Sur (14, 7), elle a renvoyé tranquillement un résultat faux. C'est le pire des cas — et c'est celui que les tests de propriété visent à rendre bruyant.

02 L'objectif et la démarche

diapo 5

OBJECTIF DU COURS

Étant donnée une fonction, on souhaite répondre à la question : **la fonction effectue-t-elle bien ce qu'on attend qu'elle fasse ?**

Pour y répondre, on propose de procéder en **deux étapes** :

1. **décrire précisément ce que l'on attend** d'une fonction : le **contrat de fonction**, ou **spécification** ;
2. **fournir un outil pour vérifier le comportement** de la fonction : les **tests de propriété**.

C'est la même question qu'au CM 1 — mais la réponse change de nature. Les tests unitaires vérifiaient la fonction sur des *exemples choisis à l'avance* ; ici on va écrire ce qui doit être vrai **à chaque appel, quels que soient les paramètres**, et laisser le programme le vérifier lui-même.

03 Pré-condition et post-condition

diapo 6

Afin de caractériser précisément ce que l'on attend d'une fonction, il est nécessaire de décrire les **exigences** que l'on se donne sur les **paramètres** et sur les **valeurs de retour** de la fonction étudiée.

DÉFINITIONS

- On appelle **pré-condition** la condition devant être vérifiée par les **paramètres** de la fonction afin que le problème puisse être résolu.
- On appelle **post-condition** la condition devant être vérifiée par la (les) **valeur(s) de retour** de la fonction. Cette vérification fait intervenir les paramètres de la fonction.

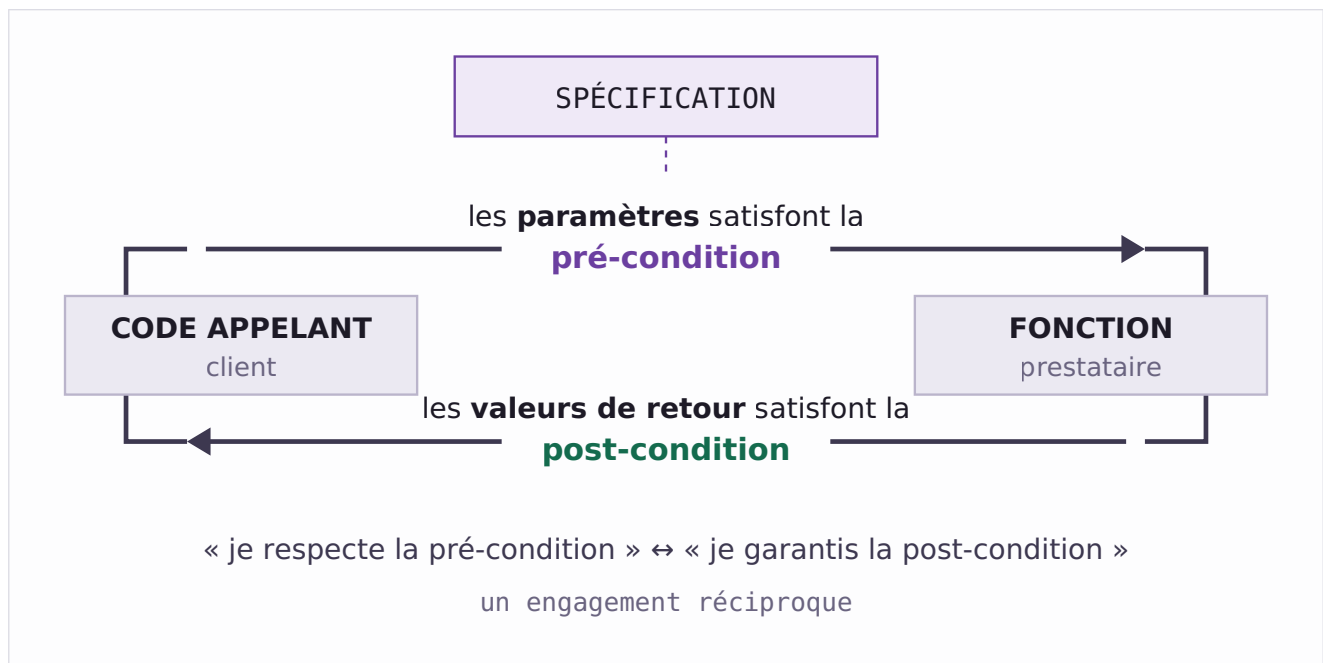
Ces deux conditions sont écrites sous forme d'**expressions booléennes**.

Deux détails de cette définition sont souvent perdus en examen, et pourtant décisifs :

- la post-condition **fait intervenir les paramètres**. Dire « le résultat est un entier » ne dit rien ; il faut relier le résultat à ce qui est entré — $a == q*b + r$;
- ce sont des **expressions booléennes**, donc du code exécutable, pas du commentaire. C'est ce qui permettra la partie III.

04 Le contrat de fonction

diapos 7–8



Le vocabulaire du cours est celui du commerce : le **client** (l'utilisateur de la fonction) et le **prestataire** (son codeur). Chacun s'engage sur une moitié du contrat.

DÉFINITION

On appelle **spécification** d'une fonction la donnée :

- du **type** des paramètres et de la valeur de retour ;
- de la **pré-condition** et de la **post-condition** de cette fonction.

La spécification constitue le **contrat qui lie le codeur de la fonction (prestataire) avec son utilisateur (client)**. Le codeur s'engage, **dès lors que l'utilisateur respecte la pré-condition**, à respecter la post-condition. On parle encore de **contrat de fonction**.

La clause « dès lors que » est le cœur de l'affaire. Si le client appelle `division_euclidienne(10, 0)` alors que la pré-condition exige $b > 0$, le prestataire n'est plus tenu à rien : le comportement de la fonction devient un problème du client, pas du codeur.

05 Spécification et implémentation

diapo 9

DÉFINITION

On appelle **implémentation** d'une spécification donnée **toute fonction qui respecte cette spécification**.

- la spécification précise le **quoi**, tandis que l'implémentation précise le **comment** ;
- la spécification est la **seule chose à savoir pour pouvoir utiliser la fonction**.

Le mot *toute* a son importance : une spécification admet en général **plusieurs implémentations**. La division euclidienne peut se calculer par soustractions successives, par l'opérateur `//`, ou par un algorithme plus rapide — ce sont trois implémentations de la même spécification, et le client n'a pas à savoir laquelle il utilise.

On retrouve la règle de nommage du CM 1 : un nom de fonction reflète **ce qu'elle fait**, pas comment. La spécification est cette idée poussée jusqu'à l'exécutable.

06 Spécifier la division euclidienne

diapo 10

Question 2. On s'intéresse à la division euclidienne de **deux entiers positifs**. a) Écrire la spécification de `division_euclidienne` dans ce cas. b) Écrire une implémentation de cette spécification.

PROPOSITION DE CORRECTION · QUESTION 2A

ÉLÉMENT	CONTENU
Type des paramètres	$a : \text{int}, b : \text{int}$
Type du retour	$(q, r) : \text{couple d'int}$
Pré-condition	$a \geq 0 \text{ and } b > 0$
Post-condition	$a == q*b + r \text{ and } 0 \leq r < b$

$b > 0$ et non $b \geq 0$: c'est ce qui exclut le cas $(6, 0)$ de la question 1, et c'est aussi ce qui rend la post-condition $0 \leq r < b$ satisfaisable.

PROPOSITION DE CORRECTION · QUESTION 2B

```
def division_euclidienne(a, b):
    q, r = 0, a
    while r >= b:
        q, r = q+1, r-b
    return q, r
```

Une seule modification par rapport au code de départ — $\geq$ au lieu de $>$ — et la boucle s'arrête exactement quand $r < b$, c'est-à-dire quand la post-condition devient vraie.

Question 3 : et pour les entiers relatifs ?

PROPOSITION DE CORRECTION

CONDITION	ENTIERS POSITIFS	ENTIERS RELATIFS
Pré-condition	$a \geq 0 \text{ and } b > 0$	$b \neq 0$
Post-condition	$a == q*b + r \text{ and } 0 \leq r < b$	$a == q*b + r \text{ and } 0 \leq r < \text{abs}(b)$

- la contrainte sur a disparaît : tout entier est divisible ;
- celle sur b s'affaiblit en $b \neq 0$;
- dans la post-condition, b est remplacé par $\text{abs}(b)$ — l'encadrement du reste doit rester un intervalle non vide, ce que $0 \leq r < b$ ne serait plus pour b négatif.

L'implémentation par soustractions successives, elle, ne convient plus telle quelle : c'est un bon rappel que **changer la spécification oblige à revisiter l'implémentation**.

REMARQUE

La convention « reste positif » retenue ici est la convention mathématique usuelle. Les opérateurs `//` et `%` de Python suivent une autre convention (le reste prend le signe de b) : deux post-conditions différentes, donc deux spécifications différentes, pour un même nom d'opération.

07 Qui perd au contrat ?

diapo 11

Question 4. Qui est le perdant du contrat dans les deux cas suivants ?

CAS A) · PRÉ-CONDITION
 $a \geq 0$ and $b \geq 0$

CAS A) · POST-CONDITION
 $a == q*b + r$ and $0 \leq r < b$

PROPOSITION DE CORRECTION · CAS A)

Le prestataire est le perdant. La pré-condition autorise $b = 0$; or pour $b = 0$ la post-condition exige $0 \leq r < 0$, ce qui est **impossible**. Le codeur s'est engagé sur une promesse qu'aucune implémentation ne peut tenir : le client a le droit d'appeler `division_euclidienne(10, 0)` et de réclamer un résultat correct.

CAS B) · PRÉ-CONDITION
 $a \geq 0$ and $b > 0$

CAS B) · POST-CONDITION
 $a == q*b + r$

PROPOSITION DE CORRECTION · CAS B)

Le client est le perdant. La post-condition est trop faible : elle ne dit rien du reste. Un prestataire paresseux peut renvoyer $q = 0$ et $r = a$ — l'égalité $a == 0*b + a$ est vraie, le contrat est **honoré à la lettre**, et le client reçoit un résultat inutile sans aucun recours.

PRÉ-CONDITION TROP FAIBLE

pré (peu exigeante) post (exigeante)

le codeur promet plus
qu'il ne peut tenir

le PRESTATAIRE perd

POST-CONDITION TROP FAIBLE

pré (exigeante) post (peu exigeante)

le codeur peut tenir sa
promesse sans rien faire

le CLIENT perd

Un bon contrat est un **équilibre** : une pré-condition assez forte pour que le problème soit résoluble, une post-condition assez forte pour que le résultat soit utile. Relâcher l'une ou l'autre fait toujours un perdant.

08 Principe et mise en place

diapos 12–13

PRINCIPE ET INSTRUMENTATION

- le principe des **tests de propriété** est de vérifier **lors de l'exécution** certaines **propriétés requises** ;
- la vérification est effectuée par des **assertions** (instruction `assert`).

Les assertions destinées à vérifier la pré-condition et la post-condition sont placées respectivement **en tout début** et **en toute fin** de la fonction testée :

STRUCTURE IMPOSÉE

```
def ...(...,...):
    assert ..., 'Pre-condition'
    ... # CODE DE LA FONCTION
    assert ..., 'Post-condition'
    return ...
```

Question 5 : compléter le code de la question 2b

PROPOSITION DE CORRECTION

```
def division_euclidienne(a, b):
    assert a >= 0 and b > 0, 'Pre-condition'
    q, r = 0, a
    while r >= b:
        q, r = q+1, r-b
    assert a == q*b + r and 0 <= r < b, 'Post-condition'
    return q, r
```

Les deux conditions écrites en français à la section 06 sont recopiées telles quelles : c'est tout l'intérêt de les avoir formulées comme des **expressions booléennes**.

09 Deux règles d'écriture

diapos 14–15

Règle 1 · les paramètres doivent rester constants

PARAMÈTRES CONSTANTS

Les paramètres de la fonction **ne doivent pas être modifiés**, sinon la post-condition ne peut pas être vérifiée correctement.

EXEMPLE INCORRECT

```
def division_euclidienne(a,b):
    assert a>=0 and b>0, 'Pre-condition'
    a,q,r = 0,0,0
    assert a==q*b+r and 0<=r<b, 'Post-condition'
    return q,r
```

La dernière assertion est **bien vérifiée** — $0 == 0*b + 0$ — mais elle **n'est pas pertinente** : en écrasant `a`, on a effacé la donnée à laquelle le résultat devait être comparé. La post-condition ne parle plus de la question posée.

Exception : les tableaux, vus plus tard dans le cours.

Règle 2 · une seule instruction `return`

UNE SEULE INSTRUCTION RETURN

La fonction ne doit présenter qu'une seule instruction `return`, située **directement après la post-condition**.

EXEMPLE INCORRECT

```
def division_euclidienne(a,b):
    assert a>=0 and b>0, 'Pre-condition'
    q,r = 0,a
    while True:
        q,r = q+1,r-b
        if r<b:
            return q,r # Post-condition non garantie
    assert a==q*b+r and 0<=r<b, 'Post-condition'
    return q,r
```

Le `return` placé dans la boucle **court-circuite l'assertion** : la fonction sort sans jamais passer par la vérification. La règle n'est donc pas une coquetterie de style — c'est la condition pour que le test de propriété soit réellement exécuté à chaque appel.

C'est aussi une bonne discipline en dehors de ce cours : une fonction avec un seul point de sortie garantit qu'un traitement de fin (libération, journalisation, vérification) ne peut pas être sauté.

10 Précisions sur assert

diapos 16-17

VÉRIFICATION STATIQUE DES TYPES

Elle **n'est pas effectuée** par les tests de propriété. Elle relève d'autres dispositifs : les **annotations de type** en Python, ou la **compilation** dans les langages compilés. Le type fait partie de la spécification, mais il n'est pas vérifié par les assertions.

S'IL N'Y A AUCUNE PRÉ-CONDITION

On l'écrit tout de même, pour que le lecteur sache que la question a été posée :

```
# Pas de pre-condition
```

ou bien :

```
assert True, 'Pre-condition'
```

RAPPELS DU CM 1 SUR LES ASSERTIONS

- elles peuvent être **coûteuses en temps et/ou en mémoire** ;
- elles sont **exclusivement réservées au débogage** ;
- elles sont **désactivées** avec le mode optimisé de Python : `python3 -O fichier.py` ;
- elles doivent donc utiliser des expressions booléennes **sans effets de bord** ;
- elles ne doivent être utilisées **ni pour valider des entrées utilisateurs, ni pour valider le contenu de fichiers**.

Ce dernier rappel se combine bien avec la notion de contrat : une pré-condition **n'est pas une validation d'entrée**. Elle ne protège pas contre un utilisateur malveillant, elle documente et vérifie une hypothèse entre deux morceaux de code.

11 Le format des examens CC3 et CC4

diapos 18-20

Si l'exercice proposé en examen est d'écrire la pré-condition et la post-condition d'une fonction donnée, les tests de propriété seront placés dans des **fonctions dédiées** :

STRUCTURE DE L'EXERCICE

```
def verifie_pre_condition(..., ...):
    return ... # Pre-condition

def verifie_post_condition(..., ...):
    return ... # Post-condition

def ...(..., ...):
    assert verifie_pre_condition(..., ...), 'Pre-condition'
    ... # CODE DE LA FONCTION
    assert verifie_post_condition(..., ...), 'Post-condition'
    return ...
```

L'EXEMPLE COMPLET DONNÉ PAR LE COURS

```

# Fonctions à compléter
def verifie_pre_condition(a,b):
    return a>=0 and b>0

def verifie_post_condition(a,b,q,r):
    return a==q*b+r and 0<=r<b

# Fonction fournie lors de l'examen
def division_euclidienne(a,b):
    assert verifie_pre_condition(a,b), 'Pre-condition'
    q, r = 0, a
    while r >= b:
        q, r = q+1, r-b
    assert verifie_post_condition(a,b,q,r), 'Post-condition'
    return q, r

```

Remarquez la signature de `verifie_post_condition` : elle prend **les paramètres** *et* **les valeurs de retour**. C'est la traduction directe de la définition — la post-condition fait intervenir les paramètres.

LE CODE DE LA FONCTION...	... SERA
<code>verifie_pre_condition</code>	à compléter
<code>verifie_post_condition</code>	à compléter
<code>division_euclidienne</code>	fourni
<code>test_verifie_pre_condition</code>	non fourni
<code>test_verifie_post_condition</code>	non fourni
<code>test_division_euclidienne</code>	fourni

Ce mécanisme sera travaillé en TP. Lecture du tableau : à l'examen, on vous demande d'écrire **les deux conditions**, et ce sont les tests unitaires fournis (au sens du CM 1) qui vérifieront que vous les avez bien écrites.

12 Maximum d'une liste

diapo 22

On souhaite écrire et valider avec des tests de propriété une fonction `calculer_maximum` qui accepte en paramètre une liste d'entiers et retourne le maximum `m` de cette liste.

Question 1 : les deux fonctions auxiliaires

Elles servent justement aux tests de propriété ; aucune pré-condition ni post-condition n'est demandée pour elles.

PROPOSITION DE CORRECTION · A) EST_MEMBRE

Retourne le booléen indiquant si `m` est un élément de `liste`.

```
def est_membre(liste, m):
    trouve = False
    for valeur in liste:
        if valeur == m:
            trouve = True
    return trouve
```

Un seul `return`, conformément à la règle 2. L'idiome Python `return m in liste` convient tout aussi bien.

PROPOSITION DE CORRECTION · B) EST_MAJORANT

Retourne le booléen indiquant si `m` est supérieur ou égal à tous les éléments de `liste`.

```
def est_majorant(liste, m):
    majorant = True
    for valeur in liste:
        if valeur > m:
            majorant = False
    return majorant
```

Question 2 : spécification et implémentation

PROPOSITION DE CORRECTION · A) LA SPÉCIFICATION

ÉLÉMENT	CONTENU
Type du paramètre	<code>liste</code> : liste d'int
Type du retour	<code>m</code> : int
Pré-condition	<code>len(liste) > 0</code>
Post-condition	<code>est_membre(liste, m) and est_majorant(liste, m)</code>

La pré-condition est indispensable : le maximum d'une liste vide n'existe pas, donc le problème ne peut pas être résolu. La post-condition, elle, dit exactement ce qu'est un maximum : **un élément de la liste qui majore la liste**.

PROPOSITION DE CORRECTION · B) L'IMPLÉMENTATION

```
def calculer_maximum(liste):
    assert len(liste) > 0, 'Pre-condition'
    maximum = liste[0]
    for valeur in liste:
        if valeur > maximum:
            maximum = valeur
    assert est_membre(liste, maximum) and est_majorant(liste, maximum), \
        'Post-condition'
    return maximum
```

Le paramètre `liste` n'est jamais modifié (règle 1), et la fonction n'a qu'un seul `return`, placé après la post-condition (règle 2).

Regardez ce que la post-condition **ne fait pas** : elle ne recalcule pas le maximum. Si elle le faisait, on testerait un calcul par le même calcul — et un bug identique des deux côtés passerait inaperçu. Elle vérifie une **propriété caractéristique** du résultat, ce qui est tout l'art de la post-condition.

13 Pourquoi les deux parties sont nécessaires

diapo 23

Question 3. Analyser pourquoi chaque partie de la post-condition est nécessaire, en exhibant à chaque fois un contre-exemple.

A) SANS EST_MEMBRE(LISTE, M)

Il ne reste que « `m` majore la liste ». N'importe quelle valeur assez grande convient alors, sans être le maximum :

LISTE	M PROPOSÉ	EST_MAJORANT	EST_MEMBRE	MAXIMUM ?
[1, 2, 3]	10	True	False	non — 10 n'est pas dans la liste

C'est `est_membre` qui interdit de renvoyer un majorant arbitraire.

B) SANS EST_MAJORANT(LISTE, M)

Il ne reste que « m appartient à la liste ». N'importe quel élément convient alors :

LISTE	M PROPOSÉ	EST_MEMBRE	EST_MAJORANT	MAXIMUM ?
[1, 2, 3]	1	True	False	non — 2 et 3 sont plus grands

C'est `est_majorant` qui interdit de renvoyer le premier élément venu.

CONCLUSION

Les deux propriétés prises ensemble **caractérisent** le maximum : aucune valeur autre que le maximum ne peut satisfaire les deux à la fois. Une post-condition à laquelle il manque une clause est une post-condition trop faible — et l'on retombe exactement sur le cas b) de la section 07 : **c'est le client qui perd**.

14 Comparaison avec les autres approches

diapos 25–27

Face aux tests unitaires

	TESTS DE PROPRIÉTÉ	TESTS UNITAIRES
Para-mètres	quelconques , non décidés à l'avance	choisis dans les fonctions de test
Moment	en contexte : lorsque la fonction est appelée par d'autres fonctions, dans un code volumineux	à l'exécution de la fonction de test
Force	vérifient à chaque appel réel	peuvent cibler spécifiquement des situations : cas extrêmes, particuliers — par exemple le 366 ^e jour d'une année bissextile
Nom anglais	<i>Property Based Testing</i>	<i>Example Based Testing</i>

Les deux sont **complémentaires** : les tests unitaires vont chercher les cas tordus auxquels personne ne pense ; les tests de propriété surveillent tout le reste, en permanence, sur des valeurs que personne n'avait prévues.

Face à la vérification formelle

	TESTS DE PROPRIÉTÉ	VÉRIFICATION FORMELLE
Nature	vérification dynamique , lors de l'appel et de l'exécution	analyse statique du code de la fonction
Vue sur le code	le code est une boîte noire : seul compte le résultat obtenu	on analyse le code lui-même
Portée	nombre fini de combinaisons testées	preuve que pour toutes les valeurs possibles des paramètres, le résultat retourné est correct
Où ?	ce cours	UE Info2.Algo2

DIJKSTRA

« *Program testing can be used to show the presence of bugs, but never to show their absence!* » — le test montre la présence de bugs, jamais leur absence.

Face à la documentation de code

	TESTS DE PROPRIÉTÉ	DOCUMENTATION (<i>DOCSTRING</i>)
Nature	mise en œuvre de la description : test effectué à l'exécution, sur des valeurs réelles	description de la fonction, du module ou de la classe, ajoutée en première ligne du bloc
Public	le programme lui-même	le développeur
Cohé- rence	le code et la vérification sont cohérents : on ne peut pas changer l'un sans changer l'autre	gérée au même endroit que le code qui l'implémente ; les exemples peuvent servir de tests unitaires via le module <code>doctest</code>

L'argument de cohérence est le plus fort en faveur des assertions : un commentaire peut mentir pendant des années sans que rien ne le signale, une assertion fausse arrête le programme dès le premier appel.

15 Un double objectif

diapo 28

- **Une méthode de débogage** : les tests de propriété permettent d'identifier **quelle portion de code a failli**. Ils s'utilisent en **complément des tests unitaires**.
- **Un pas vers la vérification formelle** : les fonctions utilisées ponctuellement pour vérifier que la fonction a correctement répondu à la question reprennent, pour la plus grande partie, les **caractérisations logiques** (formules, etc.) utilisées lors de la vérification formelle.

Le premier point est immédiat : dans un programme de plusieurs milliers de lignes, une `AssertionError` vous indique la fonction, la ligne, et si c'est la pré- ou la post-condition qui a lâché — donc si le fautif est l'appelant ou l'appelé. C'est une information que n'apporte aucun affichage de résultat faux.

→ Glossaire des définitions

Pré-condition diapo 6	Condition devant être vérifiée par les paramètres de la fonction afin que le problème puisse être résolu.
Post-condition diapo 6	Condition devant être vérifiée par la ou les valeurs de retour ; cette vérification fait intervenir les paramètres de la fonction.
Spécification diapo 8	Donnée du type des paramètres et de la valeur de retour, de la pré-condition et de la post-condition d'une fonction.
Contrat de fonction diapo 8	Autre nom de la spécification : le codeur (prestataire) s'engage, dès lors que l'utilisateur (client) respecte la pré-condition, à respecter la post-condition.
Implémentation diapo 9	Toute fonction qui respecte une spécification donnée. La spécification précise le quoi, l'implémentation le comment.
Test de propriété diapo 12	Vérification, lors de l'exécution, de certaines propriétés requises, effectuée par des assertions. En anglais : <i>Property Based Testing</i> .
Paramètres constants diapo 14	Règle : les paramètres ne doivent pas être modifiés, sinon la post-condition ne peut pas être vérifiée correctement. Exception : les tableaux.
Règle du return unique diapo 15	La fonction ne doit présenter qu'une seule instruction <code>return</code> , située directement après la post-condition.
Vérification formelle diapo 26	Analyse statique du code fournissant une preuve que, pour toutes les valeurs possibles des paramètres, le résultat retourné est correct. UE Info2.Algo2.
Docstring diapo 27	Description de la fonction, du module ou de la classe, ajoutée en première ligne du bloc ; ses exemples peuvent servir de tests unitaires via <code>doctest</code> .

→ Synthèse

Le chapitre tient en une chaîne : **on écrit ce qu'on attend (spécification), on l'écrit sous forme booléenne (pré- et post-condition), et on la fait vérifier par le programme lui-même (assertions)**. Le contrat cesse d'être une intention pour devenir du code qui se défend tout seul.

QUESTION	RÉPONSE DU COURS
Qu'est-ce qu'une pré-condition ?	Ce que les paramètres doivent vérifier pour que le problème soit résoluble.
Qu'est-ce qu'une post-condition ?	Ce que le résultat doit vérifier, en faisant intervenir les paramètres.
Que contient une spécification ?	Les types, la pré-condition et la post-condition. Rien de plus, rien de moins.
Où place-t-on les assertions ?	Pré-condition en tout début, post-condition en toute fin, juste avant l'unique <code>return</code> .
Pré-condition trop faible ?	Le prestataire perd : il promet l'impossible.
Post-condition trop faible ?	Le client perd : le résultat peut être inutile.
Propriété ou unitaire ?	Paramètres quelconques en contexte / paramètres choisis pour cibler des cas particuliers. Complémentaires.

LE RÉFLEXE D'EXAMEN

Avant de rendre une post-condition, posez-vous la question de la section 13 : **existe-t-il une valeur fausse qui la satisfait quand même ?** Si oui, il manque une clause. C'est le test qui distingue une post-condition correcte d'une post-condition trop faible.